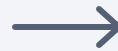


Modern C++ for control statements and functions

What C++11 to C++20 adds to the C you already know. Ten features, each with a short program you can run.

C

```
for (size_t i = 0; i < n; i++)  
    printf("%d\n", pids[i]);
```



Modern C++

```
for (int pid : pids)  
    std::cout << pid << '\n';
```

Programs 01_range_for.cpp to 10_nodiscard_optional.cpp, built with `g++ -std=c++20 -Wall -Wextra`

From C to modern C++

Each row is one slide and one program.

In C	In modern C++	Since	Program
■ Control statements			
<code>for (i = 0; i < n; i++)</code>	<code>for (int x : v)</code>	C++11	01_range_for
<code>declare, then if / switch</code>	<code>if (init; cond)</code>	C++17	02_if_switch_init
<code>enum, silent fall-through</code>	<code>enum class, [[fallthrough]]</code>	C++11, C++17	03_switch_enum
<code>goto cleanup</code>	<code>destructors (RAII)</code>	C++98	04_raii_cleanup
■ Functions			
<code>pointer, pointer + length</code>	<code>T&, const T&, std::span</code>	C++98, C++20	05_references_span
<code>one name per function</code>	<code>overloading, default arguments</code>	C++98	06_overload_default
<code>one copy per type</code>	<code>auto, templates</code>	C++14, C++20	07_auto_generic
<code>function pointer + void*</code>	<code>lambdas</code>	C++11	08_lambdas
<code>#define macros</code>	<code>constexpr functions</code>	C++11	09_constexpr
<code>return -1 and hope</code>	<code>[[nodiscard]], std::optional</code>	C++17	10_nodiscard_optional

```
$ g++ -std=c++20 -Wall -Wextra 01_range_for.cpp -o 01_range_for
```

Range-based for: loop over the elements

No index, no bound, no off-by-one. Works on arrays and all standard containers.

program 01_range_for.cpp

C

```
int pids[] = {101, 202, 303};
size_t n = sizeof pids / sizeof pids[0];

for (size_t i = 0; i < n; i++)
    printf("%d\n", pids[i]);

for (size_t i = 0; i < n; i++)
    pids[i] += 1000;
```

Modern C++

```
std::vector<int> pids = {101, 202, 303};

for (int pid : pids)                // a copy of each element
    std::cout << pid << '\n';

for (int& pid : pids)                // change in place
    pid += 1000;

std::map<std::string, int> procs =
    {{"init", 1}, {"sshd", 845}};
for (const auto& [name, pid] : procs) // C++17
    std::cout << name << " has pid " << pid << '\n';
```

int x copies each element, **int& x** changes it,
const auto& x reads big ones without copying.

Structured bindings (C++17) give names to the
parts of a pair or struct.

Careful: don't add or remove vector elements
inside the loop.

if and switch with an initializer

Declare a variable that exists only for that one statement.

program 02_if_switch_init.cpp

C

```
int fd = open("/etc/hostname", O_RDONLY);
if (fd >= 0) {
    /* read ... */
    close(fd);
} else {
    perror("open");
}
/* fd is still visible here */

pid_t pid = fork();
switch (pid) {
    /* ... */
}
```

Modern C++

```
if (int fd = open("/etc/hostname", O_RDONLY); fd >= 0) {
    // read ...
    close(fd);
} else {
    perror("open");           // fd visible here too
}
// fd does not exist here

switch (pid_t pid = fork(); pid) {
case -1:  perror("fork"); return 1;
case 0:   /* child */      _exit(0);
default:  waitpid(pid, nullptr, 0);
}
```

Syntax: `if (init; condition)` and `switch (init; value)`.

The variable is visible in **every branch**, including else, and nowhere after.

Great for lookups: `if (auto it = m.find(k); it != m.end())`

A safer switch

enum class and `[[fallthrough]]` let the compiler catch the classic switch bugs.

program 03_switch_enum.cpp

C

```
enum state { NEW, READY, RUNNING };
int x = READY;           /* silently an int */

switch (level) {
case 3: stop_services();
case 2: kill_all();       /* on purpose? */
case 1: sync_disks();
        break;
}
```

Modern C++

C++11

```
enum class State { New, Ready, Running, Blocked, Done };
// int x = State::Ready;    error: cannot convert

switch (s) {
case State::New: return "new";
// ... one case for each value
}
```

C++17, simplified

```
switch (level) {
case 3: stop_services(); [[fallthrough]];
case 2: kill_all();       [[fallthrough]];
case 1: sync_disks();     break;
}
```

enum class values are scoped (`State::Ready`) and never become `int` by accident.

With **-Wall**, a missing enum value gives warning: enumeration value 'Done' not handled in switch.

With **-Wextra**, silent fall-through warns. **[[fallthrough]]** marks it as intended.

Cleanup without goto: RAII

A destructor runs whenever an object goes out of scope, on every return path.

program 04_raii_cleanup.cpp

C: goto cleanup

```
int copy_file(const char *src, const char *dst) {
    int ret = -1, in, out;
    in = open(src, O_RDONLY);
    if (in < 0)
        goto fail;
    out = open(dst, O_WRONLY | O_CREAT, 0644);
    if (out < 0)
        goto close_in;
    /* ... copy data ... */
    ret = 0;
    close(out);
close_in:
    close(in);
fail:
    return ret;
}
```

Modern C++

simplified

```
class Fd {
public:
    explicit Fd(int fd) : fd_(fd) {}
    ~Fd() { if (fd_ >= 0) close(fd_); }
    int get() const { return fd_; }
    bool ok() const { return fd_ >= 0; }
private:
    int fd_;
};

bool copy_file(const char* src, const char* dst) {
    Fd in(open(src, O_RDONLY));
    if (!in.ok()) return false;
    Fd out(open(dst, O_WRONLY | O_CREAT, 0644));
    if (!out.ok()) return false;    // in is closed
    // ... copy data ...
    return true;

    // out, then in, closed
}
```

Destructor = cleanup label. Objects are destroyed in reverse order of creation.

The library already does this: `unique_ptr` frees, `Lock_guard` unlocks, `ifstream` closes.

C++ also forbids a **goto** that jumps over a variable's initialization.

References and std::span as parameters

Say what a function needs: a variable to change, or a view of an array.

program 05_references_span.cpp

C

```
void swap_ptr(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}

swap_ptr(&x, &y);

long sum_c(const int *data, size_t len);
sum_c(arr, 5);    /* length can be wrong */
```

Modern C++

```
void swap_ref(int& a, int& b) {
    int t = a; a = b; b = t;
}

swap_ref(x, y);    // no & needed

size_t count_spaces(const std::string& text);

long sum(std::span<const int> data) {
    long total = 0;
    for (int x : data) total += x;
    return total;
}

sum(arr);    // length comes along
sum(vec);    // vectors too
sum(std::span(arr).first(2));    // just a part
```

T& always refers to a real object: no null, no pointer arithmetic.

const T& passes big objects without copying and without allowing changes.

std::span<T> is pointer + length in one parameter. It does not own the data.

Overloading and default arguments

One name for related functions, and parameters callers can leave out.

program 06_overLoad_default.cpp

C

```
void print_size_long(long bytes);
void print_size_double(double mib);

void log_msg(const char *msg, int level);
log_msg("disk almost full", 1);
/* level must always be given */
```

Modern C++

```
void print_size(long bytes);
void print_size(double mib);

print_size(4096L);           // long version
print_size(1.5);             // double version
// print_size(4096);         error: ambiguous

void log_msg(std::string_view msg, int level = 1);
void log_msg(int error_code);

log_msg("disk almost full"); // level = 1
log_msg("disk full", 3);
log_msg(28);                 // int version
```

The compiler picks the overload from the **argument types**, at compile time.

Defaults go in the **declaration**, and only on the **last** parameters.

Overloads need different symbol names: nm shows `_Z10print_sizeL` and `_Z10print_sized`.

auto: let the compiler work out the type

For variables, return types, and since C++20, parameters.

program 07_auto_generic.cpp

C

```
unsigned long pages_needed(unsigned long b) {
    return (b + 4095) / 4096;
}

int    max_int(int a, int b);
double max_double(double a, double b);
/* one copy per type, or a macro */
```

Modern C++

```
auto pages_needed(unsigned long bytes) {      // C++14
    return (bytes + 4095) / 4096;             // unsigned long
}

auto larger(auto a, auto b) {                 // C++20
    return a > b ? a : b;
}

larger(3, 7);                                // int
larger(2.5, 1.5);                            // double
larger(std::string("bash"), std::string("zsh"));

template <typename T>                         // classic syntax
T smaller(T a, T b) { return a < b ? a : b; }

for (auto it = v.begin(); it != v.end(); ++it)
```

Still static typing: every type is fixed at compile time.

A function with auto parameters is a **template**: one version is compiled per type used.

Use auto when the type is obvious or long. Write it out when it helps the reader.

Lambdas: functions defined in place

They replace small helper functions and function pointers, and they start threads.

program 08_Lambdas.cpp

C

```
int by_priority(const void *a, const void *b)
{
    const struct proc *pa = a, *pb = b;
    return pa->priority - pb->priority;
}

qsort(procs, n, sizeof procs[0],
      by_priority);
```

Modern C++

```
std::sort(procs.begin(), procs.end(),
          [](const Proc& a, const Proc& b) {
              return a.priority < b.priority;
          });

for (int id = 1; id <= 4; ++id)
    workers.emplace_back([id, &m, &total] {
        std::lock_guard<std::mutex> lock(m);
        total += id;
    });
```

`[captures](parameters) { body }`

`[]` nothing `[x]` copy `&x` reference

Type-checked: no `void*` casts, and the compiler can inline the call.

A lambda can **capture** local variables. A C function pointer cannot.

Threads: capture by reference only what outlives the thread, and join before it goes away.

constexpr: run code at compile time

A typed, debuggable replacement for many macros.

program 09_constexpr.cpp

C

```
#define PAGES(bytes) (((bytes) + 4095) / 4096)

int table[PAGES(10000)];

/* No types, no checks.
   Remember SQUARE(1 + 2) giving 5? */
```

Modern C++

```
constexpr unsigned long pages(unsigned long bytes,
                               unsigned long page_size = 4096) {
    return (bytes + page_size - 1) / page_size;
}

constexpr unsigned long factorial(unsigned n) {
    unsigned long r = 1;
    for (unsigned i = 2; i <= n; ++i)    // C++14
        r *= i;
    return r;
}

static_assert(pages(4097) == 2);    // at compile time
std::array<int, pages(10000)> page_table{};
std::cout << pages(bytes);        // at run time too
```

constexpr functions can run at compile time when their arguments are constants.

static_assert stops the build if the condition is false.

The same function also works at run time, with normal types and debugging.

[[nodiscard]] and std::optional

Make errors hard to ignore, and visible in the function's type.

program 10_nodiscard_optional.cpp

C

```

int write_all(int fd, const char *buf,
              size_t len);
write_all(fd, buf, len); /* result ignored */

int parse_pid(const char *s); /* -1: error */
int pid = parse_pid(arg);
kill(pid, SIGTERM);
/* kill(-1, ...) signals every
   process you are allowed to */

```

Modern C++

```

[[nodiscard]] bool write_all(int fd, std::string_view s);

write_all(1, "hi\n");
// warning: ignoring return value of 'bool write_all(...)',
//          declared with attribute 'nodiscard'

std::optional<int> parse_pid(std::string_view text);

if (auto pid = parse_pid(s)) // empty optional is false
    std::cout << "pid " << *pid << '\n';
else
    std::cout << "not a valid pid\n";

int pid = parse_pid("oops").value_or(1);

```

[[nodiscard]] turns a forgotten check into a compiler warning, even without -Wall.

std::optional<T> holds a T or nothing. Unwrap it with *, -> or value().

C++23 adds **std::expected<T, E>** to carry the error too, such as an errno.

Takeaways

Loops	Prefer <code>for (const auto& x : v)</code> to index loops.
Scope	Use <code>if (init; cond)</code> to keep helper variables local.
switch	<code>enum class</code> + <code>-Wall</code> catches missing cases; mark real fall-through.
Cleanup	Let destructors do it: <code>unique_ptr</code> , <code>lock_guard</code> , your own RAII classes.
Parameters	<code>const T&</code> to read, <code>T&</code> to change, <code>std::span</code> for arrays.
Generic code	Overloads, <code>auto</code> and <code>lambdas</code> instead of copies and <code>void*</code> .
Constants	<code>constexpr</code> functions instead of macros.
Errors	<code>[[nodiscard]]</code> and <code>std::optional</code> make failures visible.

Run the examples

```
$ make
$ ./01_range_for
$ ./09_constexpr 10000
$ make clean
```

Or build one file by hand:

```
$ g++ -std=c++20 -Wall -Wextra -pthread \
    08_lambdas.cpp -o 08_lambdas
```

Needs GCC 11 or newer, or Clang 14 or newer. Try the same programs on godbolt.org to see the assembly each feature produces.